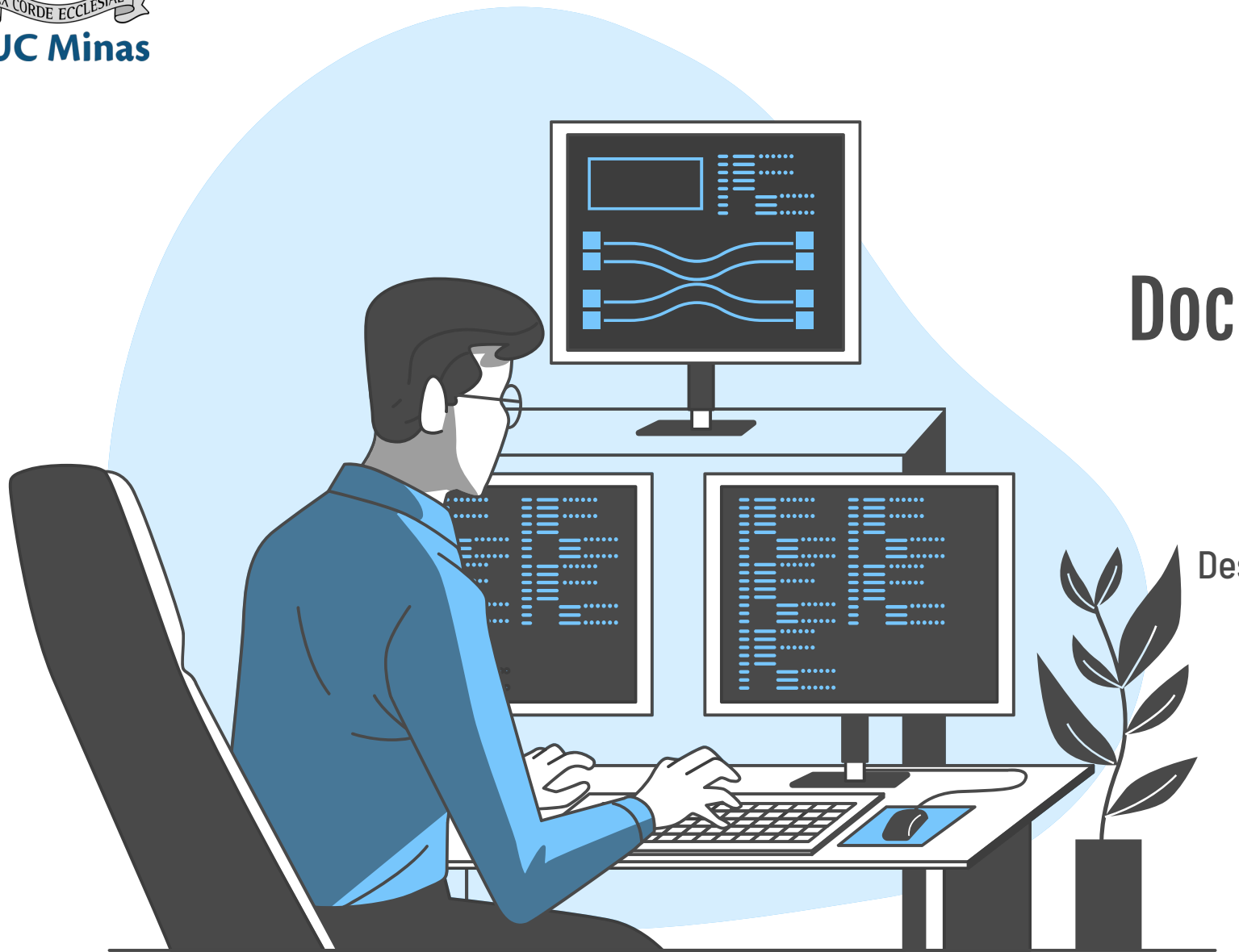
PUC Minas



Aula 03 · Deploy

# Dockerfile para Spring Boot com JDK 25

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

# Nesta aula você vai

1

Escrever um Dockerfile **multi-stage** para Spring Boot

2

Usar as imagens `maven:3.9-eclipse-temurin-25` e `eclipse-temurin:25-jre`

3

Entender cada instrução: `FROM`, `WORKDIR`, `COPY`, `RUN`, `EXPOSE`, `ENTRYPOINT`

4

Criar o `.dockerignore`

5

Construir e rodar a imagem localmente com variáveis de ambiente

# Antes de começar

- Aula 02: Docker instalado
- Back-end, aula 03 (`${VAR:padrao}`)

Duração sugerida

**100 min**

**Código da disciplina**

Projetos de referência na pasta PROJETOS do GitHub

# Roteiro

01 Por que duas etapas

02 .dockerignore

03 Construindo e rodando localmente

04 Problemas comuns

05 Aprofundando: imagens Java menores e mais seguras

01

# Por que duas etapas

# Por que duas etapas

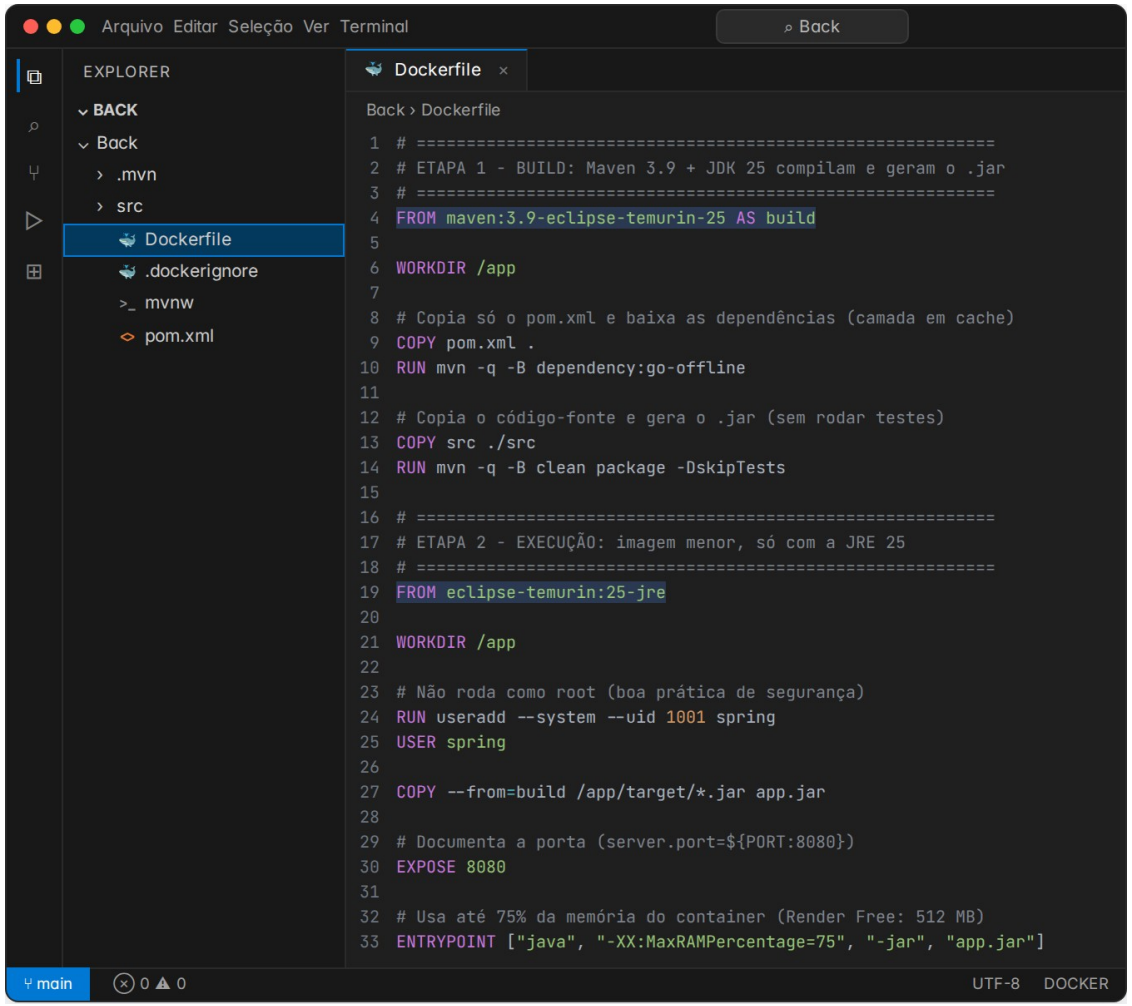
Para **compilar** precisamos de Maven e JDK (imagem de ~500 MB). Para **rodar** basta a JRE (~200 MB). O *multi-stage build* usa a primeira imagem só para gerar o `.jar` e copia apenas o `.jar` para a segunda. A imagem final fica menor, sobe mais rápido e não carrega ferramentas desnecessárias.

01

**Por que duas etapas**

Aula 03 · Deploy

# Por que duas etapas




Dockerfile do back, baseado nos projetos `Exemplo_Integracao` e `PalpiteDTI`, atualizado para Java 25.

# Por que duas etapas

| Instrução | O que faz |
|---|---|
| `FROM imagem AS nome` | Começa uma etapa a partir de uma imagem base. `AS build` dá um nome à etapa. |
| `WORKDIR /app` | Pasta de trabalho dentro da imagem (cria se não existir) |
| `COPY origem destino` | Copia arquivos do seu projeto para a imagem |
| `RUN comando` | Executa um comando **durante o build** (gera uma camada) |
| `COPY --from=build` | Copia um arquivo **de outra etapa** |
| `USER spring` | Os próximos comandos e o container rodam com esse usuário |
| `EXPOSE 8080` | Documenta a porta. Não publica nada sozinho (quem publica é o `-p`). |
| `ENTRYPOINT [...]` | O comando que roda quando o container inicia |

# As imagens do JDK 25

| Imagem | Contém | Usada em |
|---|---|---|
| `maven:3.9-eclipse-temurin-25` | Maven 3.9 + JDK Temurin 25 | Etapa de build |
| `eclipse-temurin:25-jre` | Só a JRE 25 (Ubuntu) | Etapa de execução |
| `eclipse-temurin:25-jre-alpine` | JRE 25 em Alpine Linux (menor) | Alternativa à anterior |
| `eclipse-temurin:25-jdk` | JDK completo | Se precisar de `jcmd`, `jfr` etc. |

# As imagens do JDK 25

**Atenção**

A versão do Java precisa bater em **três lugares**: `25` no `pom.xml`, a imagem de **build** (`...temurin-25`) e a de **execução** (`...:25-jre`). Se o `pom.xml` pede 25 e a imagem é 17, o build falha com *`release version 25 not supported`*. Se compilar com 25 e rodar com 17: *`UnsupportedClassVersionError … class file version 69.0`*.

# As imagens do JDK 25

### Observação

Por que `mvn` e não `./mvnw` no Dockerfile? A imagem `maven` já traz o Maven instalado. Usar `./mvnw` também funciona, mas exige copiar `.mvn/` e `mvnw` e dar permissão de execução (`RUN chmod +x mvnw`).

# Nome fixo do .jar (opcional)

`COPY --from=build /app/target/*.jar app.jar` funciona porque o Spring gera um só `.jar` executável. Se preferir um nome fixo, use `app` no ` do pom.xml, como o PalpiteDTI faz com PalpiteDTI.jar`.

01

**Por que duas etapas**

Aula 03 · Deploy

# 02

# .dockerignore

# .dockerignore

Funciona como o `.gitignore`, mas para o `COPY`. Evita enviar ao Docker pastas grandes e segredos:

Back/.dockerignore

```
target/
.idea/
.vscode/
*.iml
.git/
.env
HELP.md
```

# 03

# Construindo e rodando localmente

# Construindo e rodando localmente

Teste **sempre** antes de enviar ao Render. Se não funcionar no seu Docker, não vai funcionar lá.

Terminal

```
aluno@pc:~/projeto/Back$ docker build -t meu-back .
[+] Building 94.2s (15/15) FINISHED
 ⇒ [build 1/6] FROM docker.io/library/maven:3.9-eclipse-temurin-25
 ⇒ [build 4/6] RUN mvn -q -B dependency:go-offline                  61.3s
 ⇒ [build 6/6] RUN mvn -q -B clean package -DskipTests              24.7s
 ⇒ [stage-1 1/4] FROM docker.io/library/eclipse-temurin:25-jre
 ⇒ ⇒ naming to docker.io/library/meu-back:latest
aluno@pc:~/projeto/Back$ docker images meu-back
REPOSITORY   TAG       IMAGE ID       CREATED          SIZE
meu-back     latest    4b2e8f9a1c3d   10 seconds ago   312MB
```

Primeiro build: o download das dependências demora; os próximos usam cache.

# Construindo e rodando localmente

Terminal

```
aluno@pc:~/projeto/Back$ docker run --rm -p 8080:8080 \
    -e MONGODB_URI='mongodb+srv://aluno_app:SENHA@cluster0.ab1cd.mongodb.net/' \
    -e CORS_ORIGINS='http://localhost:5173' \
    meu-back

  .   ____          _            __ _ _
 :: Spring Boot ::                (v4.1.1)
Tomcat started on port 8080 (http) with context path '/'
Started BackApplication in 3.41 seconds (process running for 3.9)
```

Rodando a imagem com as variáveis de produção. Abra `http://localhost:8080/api/status` para conferir.

# Construindo e rodando localmente

## Dica

Simule o Render: ele define `PORT=10000`. Rode com `-e PORT=10000 -p 8080:10000` e confira que o Spring sobe na 10000. Se subir na 8080, falta o `server.port=${PORT:8080}` no `application.properties`.

# 04

# Problemas comuns

# Problemas comuns

| Erro | Causa | Solução |
|---|---|---|
| `release version 25 not supported` | Imagem de build com JDK menor | Use `maven:3.9-eclipse-temurin-25` |
| `UnsupportedClassVersionError` | Imagem de execução com Java menor | Use `eclipse-temurin:25-jre` |
| `no main manifest attribute, in app.jar` | Faltou o `spring-boot-maven-plugin` no `pom.xml` | Adicione o plugin (Back-end, aula 03) |
| `COPY failed: no source files were specified` | O `.jar` não foi gerado (build falhou antes) ou o Dockerfile está na pasta errada | Rode `docker build` dentro de `Back/` e leia o erro anterior |
| Container encerra com `OutOfMemoryError` / *exit 137* | Pouca memória | Mantenha `-XX:MaxRAMPercentage=75`; evite carregar muito na inicialização |
| Testes falham no build | `mvn package` roda os testes, que tentam conectar ao banco | Use `-DskipTests` no Dockerfile |

05

# Aprofundando: imagens Java menores e mais seguras

# Ubuntu (`25-jre`) x Alpine (`25-jre-alpine`)

## eclipse-temurin:25-jre

- Base Ubuntu, glibc
- Compatível com quase tudo
- Um pouco maior

vs

## eclipse-temurin:25-jre-alpine

- Base Alpine, musl
- Imagem menor
- Algumas bibliotecas nativas podem dar problema

# Aprofundando: imagens Java menores e mais seguras

Etapa de execução com healthcheck

```
FROM eclipse-temurin:25-jre
WORKDIR /app
RUN useradd --system --uid 1001 spring
USER spring
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s --start-period=40s \
  CMD wget -qO- http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-XX:+ExitOnOutOfMemoryError", "-jar", "app.jar"]
```

# Aprofundando: imagens Java menores e mais seguras

| Opção da JVM | Efeito |
|---|---|
| `-XX:MaxRAMPercentage=75` | Usa até 75% da memória do container |
| `-XX:+ExitOnOutOfMemoryError` | Encerra ao faltar memória (o Render reinicia limpo) |
| `-XX:+UseSerialGC` | Coletor simples, bom para containers com pouca CPU |
| `-Dspring.profiles.active=prod` | Ativa o profile de produção |
| `-XX:TieredStopAtLevel=1` | Sobe mais rápido (menos otimização JIT) |

# Alternativa: imagem sem Dockerfile

Cloud Native Buildpacks

```
# o plugin do Spring Boot gera a imagem com Buildpacks
./mvnw spring-boot:build-image -Dspring-boot.build-image.imageName=meu-back

docker run -p 8080:8080 -e MONGODB_URI=... meu-back
```

# Aprofundando: imagens Java menores e mais seguras

O `spring-boot:build-image` cria uma imagem otimizada sem você escrever o Dockerfile. É prático localmente, mas o Render constrói a partir do **Dockerfile do repositório**, por isso a disciplina mantém o Dockerfile explícito: você vê e controla cada passo.

05

**Aprofundando: imagens Java menores e mais seguras**

Aula 03 · Deploy

# Camadas do jar (layered jar)

### dependencies

Bibliotecas que quase nunca mudam.

### spring-boot-loader

Carregador do Spring Boot.

### snapshot-dependencies

Dependências em desenvolvimento.

### application

Seu código: a única camada que muda a cada commit.

# Aprofundando: imagens Java menores e mais seguras

### Dica

Rode `docker history meu-back` para ver o tamanho de cada camada da imagem. Se uma camada inesperada estiver enorme, provavelmente o `.dockerignore` deixou passar `target/` ou `.git/`.

# Aprofundando: imagens Java menores e mais seguras

1. Crie o `Dockerfile` e o `.dockerignore` dentro de `Back/`.
2. `docker build -t meu-back .` dentro de `Back/`.
3. `docker run --rm -p 8080:8080 -e PORT=8080 -e MONGODB_URI=... meu-back`.
4. Abra `/api/status` e teste um CRUD pelo front local.
5. Só então faça o push e crie o serviço no Render.

# 06

# Quiz de revisão

# Pergunta

## Quais imagens o Dockerfile do back usa com Java 25?

A node e nginx

B maven:3.9-eclipse-temurin-25 e eclipse-temurin:25-jre

C openjdk:17 e alpine

D ubuntu e java

# Resposta

## Quais imagens o Dockerfile do back usa com Java 25?

A node e nginx

✓ maven:3.9-eclipse-temurin-25 e eclipse-temurin:25-jre

C openjdk:17 e alpine

D ubuntu e java

**Por quê:** Uma para compilar, outra menor para rodar.

# Pergunta

## O erro `UnsupportedClassVersionError` indica:

A. Falta de memória

B. O jar foi compilado com Java mais novo que o da imagem de execução

C. Porta errada

D. Falta do Maven

Quiz · questão 2 de 5

# Resposta

## O erro `UnsupportedClassVersionError` indica:

A. Falta de memória

✓ O jar foi compilado com Java mais novo que o da imagem de execução

C. Porta errada

D. Falta do Maven

**Por quê:** Java 25 nos três lugares: pom, build e execução.

# Pergunta

## Por que usar `-DskipTests` no Dockerfile?

A. Para o jar ficar menor

B. Os testes podem tentar conectar ao banco, que não existe no build

C. É obrigatório

D. Para o CORS

Quiz · questão 3 de 5

# Resposta

## Por que usar `-DskipTests` no Dockerfile?

A. Para o jar ficar menor

✓ Os testes podem tentar conectar ao banco, que não existe no build

C. É obrigatório

D. Para o CORS

**Por quê:** Os testes rodam no CI (aula 09).

# Pergunta

## O que faz `-XX:MaxRAMPercentage=75`?

A. Limita a CPU

B. A JVM usa até 75% da memória do container

C. Acelera o build

D. Liga o GC

# Resposta

## O que faz `-XX:MaxRAMPercentage=75`?

A. Limita a CPU

✓ A JVM usa até 75% da memória do container

C. Acelera o build

D. Liga o GC

**Por quê:** Importante nos 512 MB do Render Free.

Quiz · questão 5 de 5

# Pergunta

## Para que serve o `.dockerignore`?

A Ignorar erros

B Não copiar target/, .git/ e segredos para a imagem

C Ignorar portas

D Pular testes

# Resposta

## Para que serve o `.dockerignore`?

A Ignorar erros

✓ Não copiar target/, .git/ e segredos para a imagem

C Ignorar portas

D Pular testes

**Por quê:** Funciona como o .gitignore para o COPY.

Fechamento

# Resumo da aula

- ✓ Multi-stage: `maven:3.9-eclipse-temurin-25` compila, `eclipse-temurin:25-jre` executa.
- ✓ Copie o `pom.xml` antes do `src` para aproveitar o cache das dependências.
- ✓ Java 25 em três lugares: `pom.xml`, imagem de build e imagem de execução.
- ✓ `.dockerignore` evita copiar `target/`, `.git/` e segredos.
- ✓ Teste local: `docker build -t meu-back .` e `docker run -p 8080:8080 -e ... meu-back`.

# Exercícios

1. Escreva o Dockerfile do seu back, construa e rode com as variáveis do seu banco.

2. Compare o tamanho da imagem usando `25-jre` e `25-jre-alpine`.

3. Rode com `-e PORT=10000` e prove que o `server.port` lê a variável.

4. Provoque o erro de versão (troque a imagem de execução para 17) e anote a mensagem.

# Para ir além

- hub.docker.com/_/eclipse-temurin
- hub.docker.com/_/maven
- docs.spring.io/spring-boot/reference/packaging/container-images/dockerfiles.html

### Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

*Links e versões citados podem ter mudado. Confira sempre a documentação oficial.*




# Obrigado!

Dúvidas sobre a aula 03?

**Prof. Dr. João Paulo Aramuni**
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

**GitHub da disciplina**
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web